

# ERP@ - College ERP System

A full-stack college ERP platform with separate Admin and Student web apps plus a Node.js/Express backend. The system manages students, teachers, sections/semesters, attendance, subjects, and marks, and can email low-attendance alerts.

---

## **\*\*Table Of Contents\*\***

- 1 Project Overview
- 2 Key Features
- 3 Architecture
- 4 Tech Stack
- 5 Repository Layout
- 6 Environment Variables
- 7 Getting Started
- 8 API Reference
- 9 Data Model Overview
- 10 Notes And Known Constraints

---

## **\*\*Project Overview\*\***

This repository contains three apps that work together.

- 1 Backend API: REST services for authentication, sections, attendance, subjects, and marks.
- 2 Admin Web App: Admin-only UI for managing teachers, sections, subjects, students, attendance, and marks.
- 3 Student Web App: Student UI for login, profile, and attendance views.

The backend uses JWT-based auth with different headers for admin, teacher, and student access. It stores data in MongoDB via Mongoose. Attendance and marks are recorded per student and per subject with section, semester, and batch rules enforced at the API level.

---

## **\*\*Key Features\*\***

## Admin

- Admin login and token validation.
- Create sections with year, batch, semester, and assigned teacher.
- Update section name or semester and propagate to students.
- Change a student to a new section and update subject list accordingly.
- Manage teachers and update passwords.
- Manage students and update passwords.

## Teacher

- Teacher login and token validation.
- Mark daily attendance with lecture counts and date.
- View all attendance for a section.
- Add subjects to a section and automatically assign to students.
- Upload or update student marks (ST1, ST2, PUT).
- Send low-attendance email alerts to students in a section.

## Student

- Student registration with photo upload and contact info.
- Student login and profile fetch.
- View attendance for a specific date.

## Email Notifications

- Low-attendance alerts are sent using Nodemailer (Gmail) with an HTML template.
- The email flow pulls section students through the backend and emails those below 75 percent.

---

## **\*\*Architecture\*\***

- Backend: Express API on `/api` with three route groups (student, teacher, admin).
- Admin UI: Vite + React app (separate folder).
- Student UI: Vite + React app (separate folder).
- Shared data layer: MongoDB collections for students, teachers, sections.

## Auth Headers

- Admin endpoints use `admintoken` header.
- Teacher endpoints use `teachertoken` header, or `admintoken` for elevated access.
- Student endpoints use `token` header.

---

## **\*\*Tech Stack\*\***

### Backend

- Node.js, Express, MongoDB, Mongoose
- JWT auth, bcrypt hashing
- Multer for image upload (in-memory)
- Nodemailer for email

### Frontend

- React 19, Vite 6
- Tailwind CSS v4
- React Router v7
- Recharts, React Datepicker, React Calendar
- React Hot Toast, Remix Icons

---

## **\*\*Repository Layout\*\***

```
ERP@ - Copy/  
Admin/           # Admin web app (Vite + React)  
  src/  
backend/         # Express API  
  config/  
  controller/  
  middleware/  
  model/  
  routes/  
frontend/        # Student web app (Vite + React)  
  src/  
readme.md
```

---

## **\*\*Environment Variables\*\***

Create the following `.env`` files with your own values. Do not commit secrets to source control.

Backend ``backend/.env``

- ``PORT``
- ``MONGOURI``
- ``JWT_SECRET``
- ``ADMIN_EMAIL``
- ``ADMIN_PASSWORD``
- ``RESEND_API_KEY`` (optional, present but not required if Nodemailer is used)
- ``GMAIL_USER``
- ``GMAIL_PASS``
- ``BACKEND_URL`` (optional, defaults to deployed URL for internal email flow)

Admin app ``Admin/.env``

- ``VITE_BACKEND_URL``

Student app ``frontend/.env``

- ``VITE_BACKEND_URL``

---

## **\*\*Getting Started\*\***

### **1 Install dependencies**

Backend

```
cd backend
npm install
```

Admin app

```
cd ../Admin
npm install
```

Student app

```
cd ../frontend
npm install
```

## 1 Run the apps

### Backend

```
cd backend
npm run server
```

### Admin app

```
cd ../Admin
npm run dev
```

### Student app

```
cd ../frontend
npm run dev
```

---

### \*\*API Reference\*\*

#### Base URL

- `http://localhost:4000/api` by default (see `PORT`)

#### Admin Routes

- `POST /loginAdmin` logs in admin.
- `GET /getAllTeacher` returns teachers, admin auth required.
- `PUT /updateTeacherPassword` updates teacher password, admin auth required.
- `PUT /updateStudentPassword` updates student password, admin auth required.
- `PUT /updateSectionorSemester` change section name or semester, admin auth required.
- `PUT /changeStudentSection` move student to another section, admin auth required.

#### Teacher Routes

- `POST /registerTeacher` create teacher, admin auth required.
- `POST /loginTeacher` teacher login.
- `POST /markAttendance` mark attendance, teacher auth required.
- `PUT /updateTeacher` assign a teacher to a section, admin auth required.
- `GET /getattendanceStudent` view attendance for a section, teacher auth required.
- `POST /addSubjects` add subject to a section, admin auth required.

- `GET /getStudentBySection` list students by section, teacher auth required.
- `POST /send-email` send low-attendance emails, teacher auth required.
- `POST /uploadMarks` upload or update marks, teacher auth required.

#### Student Routes

- `POST /registerStudent` register a student with image upload.
- `POST /loginStudent` student login.
- `GET /getProfile` get student profile, student auth required.
- `GET /getStudent` list all students, admin auth required.
- `POST /changeYear` update section year, admin auth required.
- `GET /getStudentAttendance` view attendance by date, student auth required.
- `POST /createSection` create section, admin auth required.

#### Auth Header Summary

- Admin: `admintoken: `
- Teacher: `teachertoken: `
- Student: `token: `

#### Important Request Notes

- `POST /registerStudent` uses `multipart/form-data` with `image` as file field.
- Attendance checks enforce lecture counts and assigned subjects.
- Marks upload enforces exam types `ST1`, `ST2`, `PUT` and maximum totals.

---

#### **\*\*Data Model Overview\*\***

##### Section

- `name` combines section + year + batch (example: `A2\_2025` pattern).
- `semester` enum with `Ist` through `VIIIth`.
- `subjects` stored as `Semester\_Subject` strings.
- `teacher` references a Teacher document.
- `students` is an array of Student references.

## Student

- Basic profile fields, contact info, and avatar base64.
- `subjects` list derived from section subjects.
- `attendance` array stores per-subject per-date lecture totals.
- `marks` array stores exam marks per subject and semester.

## Teacher

- Profile fields, hashed password.
- `section` references assigned sections.
- `subjects` list stores keys like `Semester\_Subject\_SectionYearBatch`.

---

## **\*\*Notes And Known Constraints\*\***

- The backend `.env` in this repository is populated. Rotate secrets and remove it before sharing publicly.
- Student photo upload is stored as base64 in MongoDB, which can grow quickly.
- Attendance rules rely on subject keys that include semester and section identifiers. Keep naming consistent when adding subjects.
- Admin and teacher tokens are not interchangeable; each endpoint expects a specific header.

---

## **\*\*License\*\***

This project appears to be for college attendance management and internal use. If you plan to distribute it, add a proper license and remove all secrets from source control.